

Лабораторная работа №1

«Продвинутый калькулятор»

Цель работы

Разработать мобильное приложение-калькулятор, которое работает не только по схеме «число → операция → число → результат», а умеет принимать и вычислять полноценные арифметические выражения с учётом приоритета операций и скобок.

1. Основная функциональность

- ввод произвольного арифметического выражения;
- операции $+$, $-$, $\times$, $\div$;
- поддержка вещественных чисел;
- поддержка скобок ();
- корректный приоритет арифметических операций;
- кнопка $=$ для вычисления всего выражения;
- C — полная очистка текущего ввода;
- $\boxtimes$ — удаление последнего символа;
- % — операция процента;
- $\pm$ — смена знака числа.

Примеры корректного поведения

$$12 + 5 \times 3 = 27$$

Умножение выполняется раньше сложения.

$$(12 + 5) \times 3 = 51$$

Скобки изменяют порядок вычисления.

2. История вычислений

Приложение должно хранить последние 10 вычислений. Для каждого элемента истории необходимо отображать выражение и полученный результат.

$$(12 + 5) \times 3 = 51$$

$$20 \div 4 + 7 = 12$$

$$15.5 \times 2 = 31$$

При нажатии на элемент истории соответствующее выражение должно снова появляться в калькуляторе и быть доступно для редактирования или повторного вычисления.

3. Память калькулятора

Кнопка	Назначение
M+	Добавить текущее значение к значению в памяти

Кнопка	Назначение
M—	Вычесть текущее значение из значения в памяти
MR	Получить значение из памяти
MC	Очистить память

4. Сохранение состояния

При изменении конфигурации интерфейса, включая поворот устройства, приложение не должно терять пользовательское состояние. Необходимо сохранять:

- текущее выражение;
- историю вычислений;
- значение памяти;
- последний результат;
- состояние ошибки.

5. Обработка ошибок

Калькулятор должен корректно обнаруживать ошибочные выражения и показывать пользователю понятное состояние ошибки. Необходимо обработать как минимум следующие ситуации:

- деление на ноль;
- неправильная расстановка или количество скобок;
- два арифметических оператора подряд;
- несколько десятичных точек в одном числе;
- попытка вычислить пустое выражение;
- выражение, заканчивающееся арифметическим оператором.

Обязательное требование: приложение не должно аварийно завершаться ни в одном из перечисленных случаев.

6. Требования к реализации

Лабораторная выполняется на одном из двух стеков: Swift для iOS или Flutter. Конкретная реализация интерфейса и внутренней архитектуры может отличаться в зависимости от выбранного стека, однако функциональные требования лабораторной одинаковы.

- вычисление должно учитывать математический приоритет операций, а не выполнять операции только слева направо;
- необходимо отделить логику вычисления выражения от отображения настолько, чтобы её можно было проверять независимо от UI;
- интерфейс должен позволять выполнить все операции, перечисленные в задании;
- ошибочный пользовательский ввод не должен приводить к падению приложения.